HW1-EAS501-GirardW

November 12, 2026

1 Homework 1 - EAS 501 - LU in Python

William Girard

Due Date: 11/23/26

[1]:
```
import numpy as np
from scipy.linalg import lu
import pandas as pd
```

[2]:
```
# construct matrix A using loops
def construct_A(A):

    # 1 to 10 (excludes 11)
    for r in range(10):
        for c in range(10):
            # shift to 1-based indexing
            i = r +1
            j = c + 1

            if(i == j and (i>=2 and j <= 9)):
                A[r, c] = -5/2
            elif(i==j==1 or i==j==10):
                A[r, c] = -29/12
            elif(i-j==1 or j-i ==1):
                A[r, c] = 4/3
            elif(i-j==2 or j-i ==2):
                A[r, c] = -1/12
            else:
                A[r, c] = 0

    return A


# create empty matrix A of size 10x10
A = np.zeros(shape=(10, 10))
A = construct_A(A)
```

```
A_df = pd.DataFrame(A, index=range(1, 11), columns=range(1, 11))
A_df
```

[2]:

	1	2	3	4	5	6	7	\
1	-2.416667	1.333333	-0.083333	0.000000	0.000000	0.000000	0.000000	
2	1.333333	-2.500000	1.333333	-0.083333	0.000000	0.000000	0.000000	
3	-0.083333	1.333333	-2.500000	1.333333	-0.083333	0.000000	0.000000	
4	0.000000	-0.083333	1.333333	-2.500000	1.333333	-0.083333	0.000000	
5	0.000000	0.000000	-0.083333	1.333333	-2.500000	1.333333	-0.083333	
6	0.000000	0.000000	0.000000	-0.083333	1.333333	-2.500000	1.333333	
7	0.000000	0.000000	0.000000	0.000000	-0.083333	1.333333	-2.500000	
8	0.000000	0.000000	0.000000	0.000000	0.000000	-0.083333	1.333333	
9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	-0.083333	
10	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	

	8	9	10
1	0.000000	0.000000	0.000000
2	0.000000	0.000000	0.000000
3	0.000000	0.000000	0.000000
4	0.000000	0.000000	0.000000
5	0.000000	0.000000	0.000000
6	-0.083333	0.000000	0.000000
7	1.333333	-0.083333	0.000000
8	-2.500000	1.333333	-0.083333
9	1.333333	-2.500000	1.333333
10	-0.083333	1.333333	-2.416667

[3]:
```
import pandas as pd

# compute LU decomposition
L, U = lu(A, permute_l=True)

# convert to dataframe for printing
# set index/columns labels to 1-10
L = pd.DataFrame(L, index=range(1, 11), columns=range(1, 11))
print('L Matrix: \n')
L
```

```
L Matrix:
```

[3]:

	1	2	3	4	5	6	7	\
1	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
2	-0.551724	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
3	0.034483	-0.729642	1.000000	0.000000	0.000000	0.000000	0.000000	
4	-0.000000	0.047231	-0.816867	1.000000	0.000000	0.000000	0.000000	
5	-0.000000	-0.000000	0.053494	-0.868654	1.000000	0.000000	0.000000	
6	-0.000000	-0.000000	-0.000000	0.057212	-0.902953	1.000000	0.000000	

```
7  -0.000000 -0.000000 -0.000000 -0.000000  0.059674 -0.927343  1.000000
8  -0.000000 -0.000000 -0.000000 -0.000000 -0.000000  0.061425 -0.945576
9  -0.000000 -0.000000 -0.000000 -0.000000 -0.000000 -0.000000  0.062735
10 -0.000000 -0.000000 -0.000000 -0.000000 -0.000000 -0.000000 -0.000000

          8         9    10
1   0.000000  0.000000  0.0
2   0.000000  0.000000  0.0
3   0.000000  0.000000  0.0
4   0.000000  0.000000  0.0
5   0.000000  0.000000  0.0
6   0.000000  0.000000  0.0
7   0.000000  0.000000  0.0
8   1.000000  0.000000  0.0
9  -0.959722  1.000000  0.0
10  0.063750 -0.971017  1.0
```

[4]:
```
U = pd.DataFrame(U, index=range(1, 11), columns=range(1,11))
print('U Matrix: \n')
U
```

```
U Matrix:
```

[4]:
```
          1         2         3         4         5         6         7   \
1  -2.416667  1.333333 -0.083333  0.000000  0.000000  0.000000  0.000000
2   0.000000 -1.764368  1.287356 -0.083333  0.000000  0.000000  0.000000
3   0.000000  0.000000 -1.557818  1.272530 -0.083333  0.000000  0.000000
4   0.000000  0.000000  0.000000 -1.456576  1.265261 -0.083333  0.000000
5   0.000000  0.000000  0.000000  0.000000 -1.396468  1.260945 -0.083333
6   0.000000  0.000000  0.000000  0.000000  0.000000 -1.356657  1.258087
7   0.000000  0.000000  0.000000  0.000000  0.000000  0.000000 -1.328348
8   0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
9   0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
10  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000

          8         9         10
1   0.000000  0.000000  0.000000
2   0.000000  0.000000  0.000000
3   0.000000  0.000000  0.000000
4   0.000000  0.000000  0.000000
5   0.000000  0.000000  0.000000
6  -0.083333  0.000000  0.000000
7   1.256055 -0.083333  0.000000
8  -1.307186  1.254535 -0.083333
9   0.000000 -1.290766  1.253356
10  0.000000  0.000000 -1.194323
```

The matricies L and U have been printed above, with proper index labeling from 1-10. In total, there are 100 elements. Note that U has the lower triangle and L has the upper triangle form, with 1's along the diagonal for L (known as Doolittle's algorithm/method). Now, I just need to get the determinant from U.

[5]:
```
import math
# get determinant for A from U by multiplying its diagonal entries
# note that U is the result of Gaussian elimination

# extract diagonals and multiply
diag_U = np.diagonal(U)
det_A = math.prod(diag_U)

print(f'Determinant of A obtained through LU decomposition and Gaussian␣
 ↪elimination is: {det_A:.3f}') # round slightly
```

```
Determinant of A obtained through LU decomposition and Gaussian elimination is:
49.065
```

[6]:
```
# double check result with numpy's built in det method
# this is a sanity check to make sure my code worked

print(f'Determinant of A through numpys method is {np.linalg.det(A):.3f}')
```

```
Determinant of A through numpys method is 49.065
```

Great, they match exactly. Showing that LU decomposition –> multiplying U's diagonals is a valid (and simple) way of finding the determinant of a matrix.